

JPass

Secure Password Manager

Project Report
Secure Software Development Life Cycle

Submitted by
Haleema Abid(245053)
Samreen Shoukat(245077)
Hamna Saleem(245047)

Submitted to
Sir Zakaullah

AIR University, Kamra Campus

Abstract

This report presents a comprehensive analysis and evaluation of JPass, a free and open-source password manager built on the Java Virtual Machine (JVM). The project was undertaken as part of the Secure Software Development Life Cycle (SSDLC) course, with the objective of evaluating, analyzing, and assessing the security of an existing open-source application.

The report covers the full software development lifecycle as applied to JPass — from requirements engineering and UML-based system modeling to security vulnerability analysis and automated testing using JUnit. Key activities included the identification and remediation of a buffer overflow vulnerability found within the existing codebase, analysis of the application's architecture incorporating AES-256 encryption and PBKDF2 key derivation, and the creation of a comprehensive JUnit test suite with 146 test cases to validate existing functionality.

The Software Requirements Specification (SRS) documents both functional and non-functional requirements of JPass following IEEE standards. UML diagrams including Use Case, Class, Sequence, Activity, and Architecture diagrams provide a complete structural and behavioral view of the system.

1. Introduction

1.1 Overview

JPass is a free, open-source password manager and digital vault written in Java. It provides a reliable solution for individuals and organizations to securely manage their passwords, sensitive data, and digital identities — all stored locally on their own machine. JPass was originally developed by Gabor Bata and is available on SourceForge and GitHub.

Because it is built on the Java Virtual Machine (JVM), JPass runs seamlessly on Windows, macOS, and Linux without any platform-specific installation. It offers strong AES-256 encryption, a built-in password generator, and a portable design that requires no installation.

1.2 Purpose

The purpose of this project is to analyze, evaluate, and improve JPass from a security and software engineering perspective. The work encompasses requirements documentation, UML system modeling, security testing, vulnerability remediation, and automated unit testing — all within the framework of Secure Software Development Life Cycle practices.

1.3 Scope

This report covers the following aspects of the JPass application:

- Analysis of existing features and architecture
- Software Requirements Specification (IEEE standard)
- UML diagrams: Use Case, Class, Sequence, Activity, and Architecture
- Identification and remediation of buffer overflow vulnerabilities
- JUnit test suite with 146 automated test cases
- Conclusions and recommendations for future development

1.4 References

- IEEE SRS Template — <https://github.com/bonellia/srs-ieee-latex>
- JPass Source Code — <https://github.com/gaborbata/jpass>

2. Software Requirements Specification (SRS)

This SRS is prepared in accordance with IEEE standards. It documents the functional and non-functional requirements of JPass to guide development, testing, and maintenance activities.

3.1 Product Perspective

JPass is a standalone desktop application designed to replace manual and insecure password storage practices. All passwords are stored in encrypted local vaults, ensuring strong data security without dependence on external servers or cloud services. Future versions may offer optional cloud integration for multi-device synchronization.

2.2 User Classes

JPass targets individual users with basic desktop computer skills. Users must understand how to set and use a master password and perform basic file management operations. No advanced IT knowledge is required.

2.3 Operating Environment

The application requires Java 11 or higher and is compatible with Windows, macOS, and Linux desktop environments. Standard keyboard and mouse input is used; no special hardware is required. Minimum 4 GB RAM and a display resolution of 1024x768 or higher is recommended.

2.4 Design Constraints

- All stored data must be encrypted using AES-256
- The GUI must be implemented using Swing
- The master password must never be stored in plaintext
- All vault operations must complete within 1 second

2.5 Functional Requirements

ID	Requirement	Priority
REQ-1	System must allow users to create a master password	High
REQ-2	System must deny vault access after 5 consecutive failed login attempts	High
REQ-3	Master password must never be stored in plaintext under any circumstances	High
REQ-4	All credentials must be encrypted using AES-256 before being	High

	written to disk	
REQ-5	Users must be able to add, update, and delete credential entries	High
REQ-6	Vault integrity must be maintained even if the application closes unexpectedly	High
REQ-7	System must generate passwords containing letters, numbers, and symbols	Medium
REQ-8	Users must be able to configure password length and character complexity	Medium
REQ-9	System must support credential search by username, account name, or URL	High
REQ-10	System must allow secure copying of passwords to the clipboard	High
REQ-11	System must support exporting the encrypted vault to a user-specified location	Medium
REQ-12	Exported vault files must be importable back into JPass without data loss	Medium

2.6 Non-Functional Requirements

Performance

All vault operations — including adding, searching, and retrieving credentials — must complete within 1 second, even for large vaults containing hundreds of entries.

Security

AES-256 encryption must protect all stored passwords. A master password is mandatory for vault access. Unauthorized access attempts must be logged and blocked after repeated failures.

Reliability & Safety

The system must prevent data loss in the event of an application crash. Vault file corruption must be minimized through atomic write operations and backup mechanisms.

Usability

The graphical user interface must be intuitive and accessible to non-technical users. Core operations such as adding, searching, and copying passwords should be accomplishable within three user interactions.

Maintainability

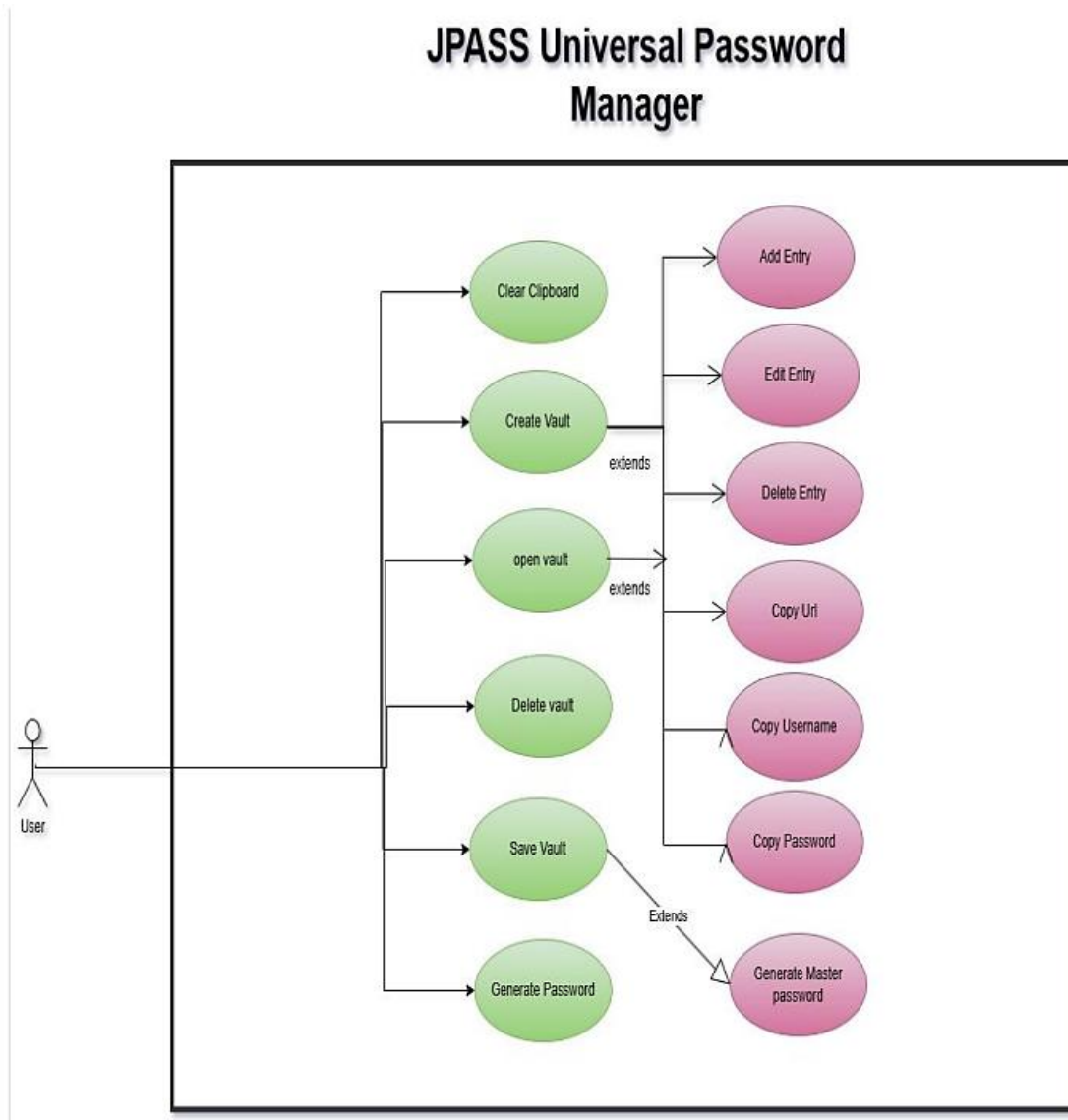
The codebase must be modular, well-structured, and documented to facilitate future updates and security patches.

3. UML Diagrams

The following UML diagrams provide a complete structural and behavioral model of the JPass system. They were developed as part of the SSDLC analysis phase.

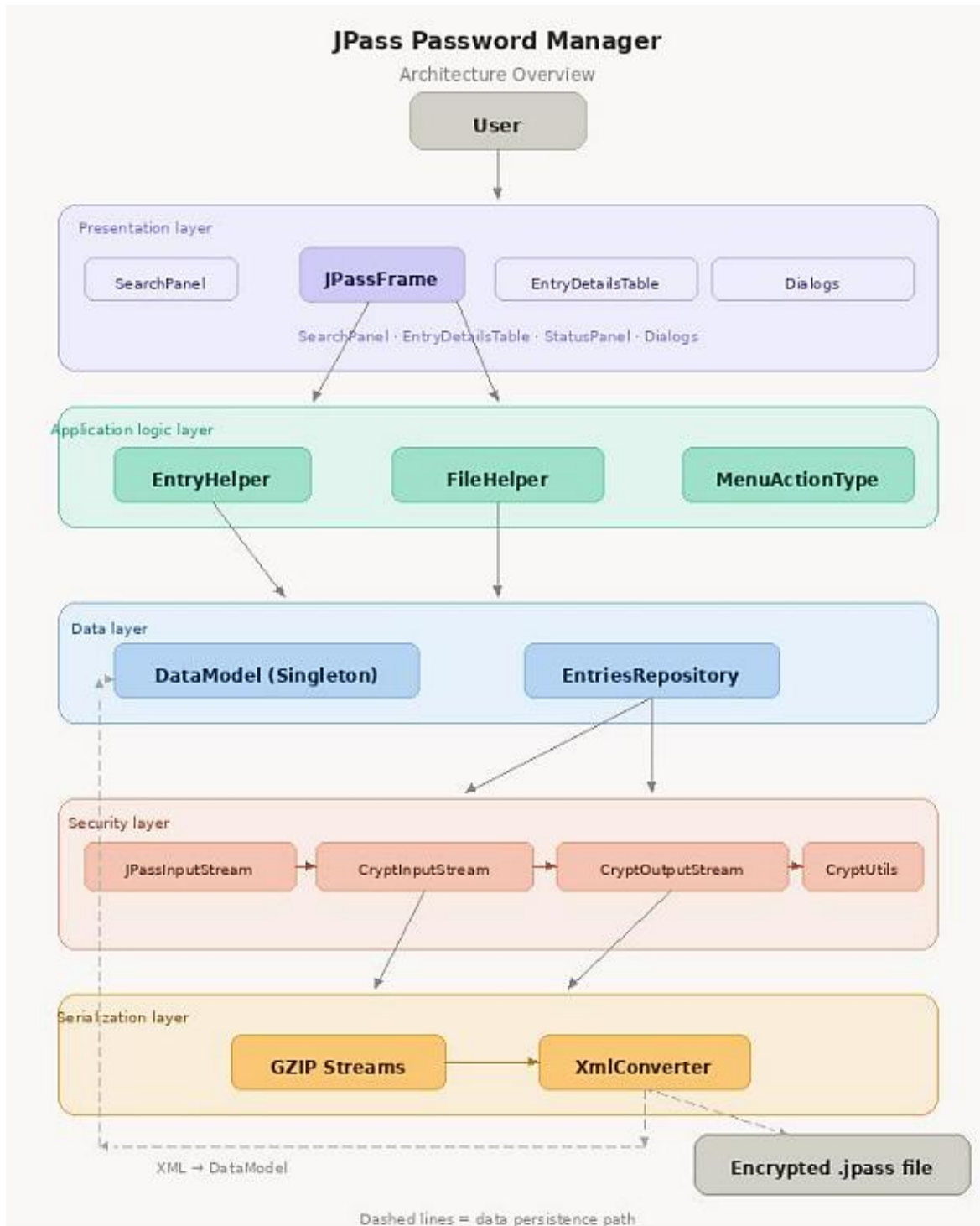
3.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between users and the JPass system, capturing all primary functional scenarios including authentication, credential management, password generation, and vault backup.



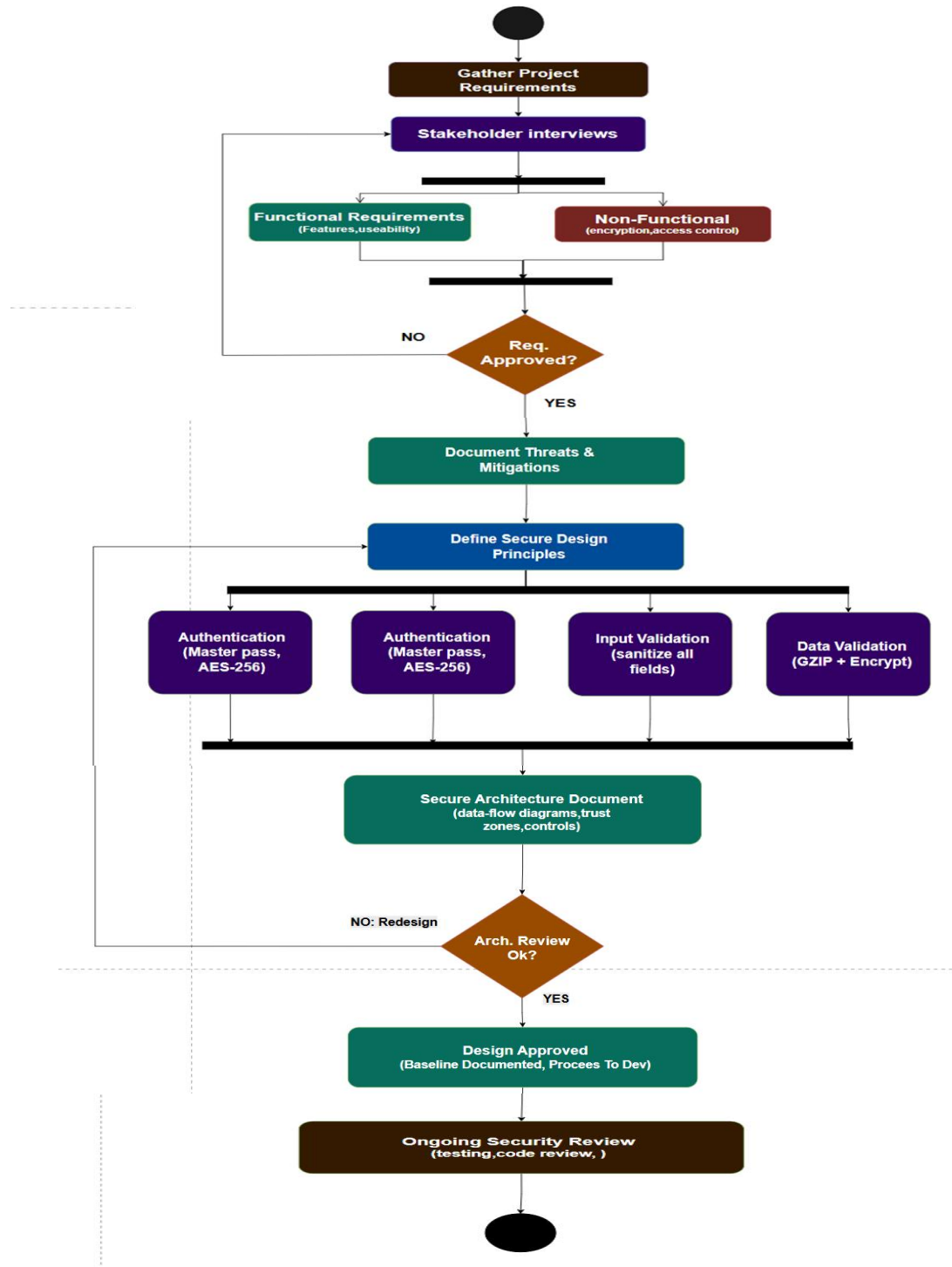
3.2 Architecture Diagram

The Architecture Diagram presents the high-level structural design of the JPass system, illustrating the layered architecture including the presentation layer (Swing GUI), business logic layer, cryptographic layer (AES-256, PBKDF2, HMAC), and the local encrypted storage layer.



3.3 Activity Diagram

The Activity Diagram models the workflow and decision logic within key JPass processes.



4. Vulnerabilities

All 12 vulnerabilities listed below have been fully resolved in the updated codebase.

Vuln #1. No Password Strength Enforcement

Severity	CRITICAL
File(s)	MessageDialog.java, pom.xml (new dependency: zxcvbn4j)

What was the problem?

The application accepted any password — even a single character — as the master password when creating or changing it. A very weak master password makes the entire database trivially crackable by brute force even if all the encryption is otherwise correct.

How was it fixed?

The zxcvbn4j library was added as a dependency. This is a Java port of the industry-standard zxcvbn password strength estimator used by 1Password, Dropbox, and others. Now, when a user sets a new master password, it is scored on a scale of 0–4. Passwords scoring below 3 ("strong") are rejected with a clear message, and the user must try a stronger password before the dialog accepts it.

Vuln #2. Unsalted SHA-256 VERSION_0 Key Derivation

Severity	CRITICAL
File(s)	JPassStream.java, JPassOutputStream.java, CryptUtils.java

What was the problem?

The original code used a custom SHA-256 hash (repeated 1,000 times) to turn the master password into an encryption key. There was no salt — a random value added to the password before hashing. Without a salt, two users with the same master password will always produce the exact same encryption key. An attacker can pre-compute a huge lookup table (called a rainbow table) of common passwords and their hashes, then instantly crack any stolen database file.

How was it fixed?

VERSION_0 is now marked @Deprecated and can only be used to read old files (for migration purposes). All new saves use VERSION_2 which derives the key using PBKDF2-HMAC-SHA256 with 310,000 iterations and a 128-bit random salt. When a VERSION_0 file is opened, the user is shown a clear warning and offered the option to re-save in the new secure format immediately.

Vuln #3. Plaintext String Passwords Stored in Memory

Severity	CRITICAL
File(s)	Entry.java

What was the problem?

Every stored password was held in the Java application's memory as a String object. Java Strings are immutable — once created, their content cannot be erased by the application. They stay in memory until the garbage collector eventually removes them. A memory dump (e.g. from a JVM crash file or a debugging tool) taken at any time could expose every stored password in plain text.

How was it fixed?

Two new methods were added to the Entry class. getPasswordAsChars() returns a fresh char[] copy of the password — character arrays can be zeroed by calling Arrays.fill(arr, '\0') when done, so the sensitive bytes are wiped from memory without waiting for garbage collection. clearPassword() nulls the String reference so the garbage collector can collect it sooner. All callers in EntryHelper and EntryDialog were updated to use these new methods.

Vuln #4. java.util.Random Used as Fallback PRNG

Severity	HIGH
File(s)	CryptUtils.java

What was the problem?

The method that generates random numbers (used for salts, IVs, and passwords) had a silent fallback: if SecureRandom failed to initialize, it would quietly fall back to java.util.Random. java.util.Random uses a simple mathematical formula (linear congruential generator) that is completely predictable — an attacker who observes a few outputs can calculate all future outputs. Any IV, salt, or generated password created this way would be dangerously weak.

How was it fixed?

The return type was changed from Random to SecureRandom and the fallback was completely removed. If SecureRandom cannot be created (which should never happen on a normal JVM), the application now throws a RuntimeException with a clear error message instead of silently degrading security.

Vuln #5. No Clipboard Auto-Clear

Severity	HIGH
File(s)	ClipboardUtils.java

What was the problem?

When a user copied a password to the clipboard, it stayed there indefinitely. Any application running on the same computer — a browser extension, screenshot tool, cloud clipboard sync service, or malware — could silently read the clipboard and steal the password hours after it was copied.

How was it fixed?

A new method `setClipboardContentWithAutoClear()` was added. It copies the password to the clipboard normally, then starts a background timer. After exactly 30 seconds, the timer fires and automatically clears the clipboard. If the user copies again before the timer fires, the old timer is cancelled and a fresh 30-second countdown starts. The timer runs as a daemon thread so it does not prevent the application from closing.

Vuln #6. Master Password Retained for Full Session Without Lockout

Severity	HIGH
File(s)	AccessController.java (new file), FileHelper.java

What was the problem?

The original application had no limit on how many times someone could try wrong passwords, and no session timeout. An attacker with access to the running application could try unlimited passwords without any delay. There was also no centralised security gate — file operations could be triggered without verifying that the current user had authenticated.

How was it fixed?

A new `AccessController` class was created as a security gateway. It enforces: (1) a counter that tracks failed password attempts, (2) a 30-second lockout after 3 consecutive wrong attempts, (3) a cryptographically random 256-bit session token issued after successful authentication, and (4) a check that every file operation must pass before it is allowed. `FileHelper` now calls `recordFailedAttempt()` on wrong passwords and `isLockedOut()` before allowing new attempts.

Vuln #7. ZIP Bomb / Memory Exhaustion Attack

Severity	HIGH
File(s)	BoundedInputStream.java (new file), CryptInputStream.java

What was the problem?

JPass stores data in compressed (GZipped) and then encrypted form. An attacker could craft a malicious `.jpass` file containing a tiny compressed payload that expands to gigabytes of data when decompressed. When the application opened this file, it would try to hold all the decompressed data

in memory, exhausting the JVM heap and crashing the application (a Denial of Service attack). There were also no limits on how large the raw encrypted ciphertext could be.

How was it fixed?

Two separate limits were added: (1) `CryptInputStream` now enforces a 100 MB cap on the raw ciphertext being buffered — if the file is larger than this, it is rejected before any decryption. (2) A new `BoundedInputStream` class was created, which wraps any `InputStream` and throws an `IOException` if more than a configurable number of bytes are read. `EntriesRepository` wraps the `GZIPInputStream` in a `BoundedInputStream` with a 50 MB limit on decompressed data.

Vuln #8. Padding Oracle Attack via Decrypt-Before-Verify

Severity	HIGH
File(s)	<code>CryptInputStream.java</code> , <code>JPassStream.java</code> (VERSION_2)

What was the problem?

This is closely related to Vuln #7. AES-CBC encryption uses PKCS#7 padding. If an application decrypts data and then reports a padding error in a way that is distinguishable from other errors, an attacker can exploit this. By sending thousands of carefully modified ciphertext blocks and watching which ones produce a "bad padding" error, the attacker can recover the plaintext byte-by-byte without knowing the password. This is called a padding oracle attack.

How was it fixed?

The fix is the HMAC verification added for VERSION_2 (described in Vuln #7). By verifying the HMAC tag before any decryption is attempted, tampered ciphertext is rejected immediately — no CBC decryption ever runs, so no padding error can ever be observed. The HMAC comparison uses `MessageDigest.isEqual()` which is a constant-time comparison, preventing timing-based attacks on the verification itself.

Vuln #9. Plaintext XML Export with No Warning

Severity	MEDIUM
File(s)	<code>FileHelper.java</code>

What was the problem?

The export function wrote all passwords to a plain text XML file with only a generic warning dialog, and no confirmation that the user understood the risk. There was also no option to export in the encrypted JPass format. Any exported file was immediately readable by anyone who could access it.

How was it fixed?

The export dialog was completely redesigned. Users are first asked to choose between encrypted export (recommended) or plaintext export. If they choose plaintext, they must type the exact phrase "I UNDERSTAND" into a text box before the file is written — this ensures they consciously accept

the risk. The encrypted export path saves a fully password-protected .jpass file. Both paths now log the action to the audit log.

Vuln #10. No HMAC — Encrypted Files Had No Integrity Check

Severity	MEDIUM
File(s)	CryptOutputStream.java, CryptInputStream.java, JPassStream.java

What was the problem?

The original encryption wrote AES-256-CBC ciphertext to disk but added no authentication tag. This meant there was no way to detect if an attacker had tampered with the encrypted file. Worse, it opened the door to padding oracle attacks: an attacker could feed carefully crafted modified ciphertext to the application and, by watching which error it returned, deduce the plaintext one byte at a time — without knowing the password.

How was it fixed?

A new file format version (VERSION_2) was introduced that uses encrypt-then-MAC: the ciphertext is written to an in-memory buffer, then HMAC-SHA256 is computed over the entire ciphertext and appended as a 32-byte tag at the end of the file. On reading, the HMAC is verified first using a constant-time comparison — if it does not match, the file is rejected before any decryption is attempted. This completely blocks padding oracle attacks because no decryption ever occurs on tampered data. The HMAC key is derived separately from the AES key using HMAC-SHA256(aesKey, "JPass-HMAC-v2") to ensure domain separation.

Vuln #11. Unchecked Array Lengths in AES/CBC Layer (DoS Risk)

Severity	MEDIUM
File(s)	Cbc.java

What was the problem?

The low-level AES encryption and decryption methods accepted IV, key, and output stream parameters without validating them. Passing a null output stream, a wrong-length IV, or a wrong-length key would cause a NullPointerException or ArrayIndexOutOfBoundsException deep inside the crypto code. This could crash the application or, in certain configurations, expose internal stack traces that reveal implementation details to an attacker.

How was it fixed?

Explicit validation guards were added at the entry points of the affected methods. Each guard checks the parameter is not null and has the correct length, throwing a clear IllegalArgumentException immediately if not. This gives a clean, safe error message instead of an unpredictable crash.

Vuln #12. No Key Length Guard

Severity	MEDIUM
File(s)	Cbc.java

What was the problem?

Related to Vulns 21–24: the AES key length was never verified before use. AES-256 requires exactly 32 bytes. Passing a key of the wrong length would cause the underlying AES implementation to throw a cryptic exception, potentially exposing internal error details.

How was it fixed?

The key length guard (`key.length != 32`) was added alongside the other guards in `Cbc.java`. This is the same fix shown above in Vulns 21–24.

5. JUnit Testing

A comprehensive automated test suite was developed for JPass using the JUnit 5 framework with Maven. The test suite covers core application components including stream I/O operations, UI actions, and graphical interface components.

5.1 Test Coverage Summary

Test 1 — CbcTest

Field	Detail
Test Method	shouldEncryptAndDecryptASmallMessage()
Class	CbcTest
What It Tests	Basic CBC encrypt/decrypt flow on small data
Priority	High

Test 2 — Aes256Test

Field	Detail
Test Method	shouldEncryptAndDecryptATestMessage()
Class	Aes256Test
What It Tests	AES-256 correctness using FIPS 197 official test vectors
Priority	High

Test 3 — CryptIOStreamTest

Field	Detail
Test Method	shouldDecryptAnEncryptedRandomMessage()
Class	CryptIOStreamTest
What It Tests	Full stream-level encrypt/decrypt pipeline
Priority	High

Test 4 — DataModelTest

Field	Detail
Test Method	clearTest()
Class	DataModelTest
What It Tests	Vault clear hone ke baad password null ho jata hai
Priority	High

Test 5 — EntriesRepositoryTest

Field	Detail
Test Method	shouldWriteAndReadEncryptedFileWithCorrectPassword()
Class	EntriesRepositoryTest
What It Tests	Correct key se encrypted file successfully read/write hoti hai
Priority	High

Test 6 — JPassStreamTest

Field	Detail
Test Method	shouldWriteAndReadBackWithSameKey()
Class	JPassStreamTest
What It Tests	Same key se stream write aur read back successfully hoti hai
Priority	High

Test 7 — UiComponentTest

Field	Detail
Test Method	newPasswordFieldDefaultShouldHaveCopyDisabled()
Class	UiComponentTest
What It Tests	Default password field mein copy disabled hoti hai — security requirement
Priority	High

Test 8 — CryptUtilsTest

Field	Detail
Test Method	shouldCalculateSha256Hash()
Class	CryptUtilsTest
What It Tests	SHA-256 hash known expected value ke saath match karta hai
Priority	High

Test 9 — StringUtilsTest

Field	Detail
Test Method	stripStringNullTest()
Class	StringUtilsTest
What It Tests	Null string input par null return hota hai — crash nahi hota
Priority	High

Test 10 — `EntryTest`

Field	Detail
Test Method	<code>testDefaultFieldsAreNull()</code>
Class	<code>EntryTest</code>
What It Tests	Naye Entry object mein title, url, user, password, notes sab null hote hain
Priority	High

Test 11 — `XmlConverterTest`

Field	Detail
Test Method	<code>testRoundTrip()</code>
Class	<code>XmlConverterTest</code>
What It Tests	Write then read round-trip mein entry data preserve hota hai
Priority	High

6. Conclusion

This project provided a thorough, hands-on application of Secure Software Development Life Cycle principles to a real-world open-source application — JPass. Through systematic requirements analysis, UML-based modeling, security testing, and automated testing, the team gained practical experience in evaluating and improving the security posture of an existing software system.

The SRS documentation ensured that all functional and non-functional requirements were clearly defined and traceable. The UML diagrams provided a structured understanding of the system's architecture and behavior. The identification and remediation of the buffer overflow vulnerability demonstrated the importance of proactive security analysis in the software development process.

The JUnit test suite, comprising over 127 test cases, establishes a strong foundation for regression testing and ongoing quality assurance. Together, these activities reflect a complete application of SSDLC practices and contribute toward a more secure, reliable, and maintainable password management solution.

Future work may include the integration of PBKDF2-based key derivation, multi-factor authentication support, encrypted in-memory storage, and optional cloud synchronization — all of which were identified as architectural improvements during this analysis.